

Desenvolvimento de Componente de Simulação (DCS)

Factory de solvers de EDO + método adaptativo Dormand–Prince 5(4)
e plugin: equação de difusão em N dimensões pelo Método das Linhas

Bruno Bertolino · Thiago Kenzo · Tom Hunt

Resumo. Este componente desacopla o simulador do método numérico usado para integrar sistemas de equações diferenciais ordinárias (EDOs). A versão original instanciava diretamente o solver de Runge–Kutta de 4ª ordem de passo fixo (RK4). Introduzimos uma *Factory* (`OdeSolverFactory`) que produz implementações de `OdeSolver_if` sob demanda, implementamos o método embutido e adaptativo **Dormand–Prince 5(4)** (`DormandPrince54OdeSolver`) como um novo solver e ligamos a seleção do método ao plugin `BioNetwork` por um campo persistido e editável. Como **extensão de funcionalidade**, adicionamos um novo plugin (`DiffusionField`) que resolve a **equação de difusão em N dimensões** pelo **Método das Linhas**: diferenças finitas no espaço transformam a EDP num sistema de EDOs, integrado pelos mesmos solvers via a mesma *Factory* — sem alterar uma linha do componente da Parte 1. Toda a lógica foi validada contra soluções analíticas, com verificação empírica das ordens de convergência e das leis de conservação. **71 verificações automatizadas, 0 falhas** (núcleos numéricos + GoogleTest); os núcleos das duas partes foram **recompilados e reexecutados** para este relatório.

Estrutura. As Seções 1–7 descrevem a Parte 1 (Factory + DP5(4)). A Seção 8 descreve a extensão de difusão N -D. As Seções 9–12 trazem reprodução, implantação, análise crítica e conclusão.

1 Contexto e objetivo

O GenESyS modela simulação de eventos discretos e, no pacote `tools`, oferece utilidades numéricas. O plugin `BioNetwork` faz *simulação contínua*: a partir de espécies (`BioSpecies`) e reações (`BioReaction`) com cinética de ação das massas, monta um sistema de EDOs (`MassActionOdeSystem`) na forma $dy/dt = f(t, y)$ e o integra ao longo do tempo. Até então a integração era feita exclusivamente por um RK4 de passo fixo, instanciado de forma rígida dentro de `BioNetwork::advanceOneStep`:

```
RungeKutta4OdeSolver solver; // método fixo em compilacao
solver.advance(system, _currentTime, dt, y0, y1); // sem outra opcao
```

Esse acoplamento impede trocar o método sem editar o cliente. O objetivo do DCS é remover esse acoplamento por meio de uma *Factory*, oferecer um segundo método de melhor relação ordem/custo (Dormand–Prince 5(4)), permitir a escolha do solver em tempo de execução e validar tudo com o próprio `BioNetwork`. A **extensão** reutiliza a mesma arquitetura para resolver uma EDP de difusão, demonstrando que a abstração de solvers serve a problemas inteiramente diferentes.

2 Requisitos funcionais e casos de uso

Cada requisito (RF) é rastreável a testes automatizados (Seção 6) e a resultados (Seções 7 e 8). Entre parênteses, o nome do caso de teste correspondente.

ID	Requisito / Caso de uso	Verificação
<i>Parte 1 — Factory + Dormand–Prince 5(4)</i>		
RF1	A Factory cria solvers de EDO sob demanda, a partir de identificador estável (enum e string).	<code>CreatesNonNull...</code> , <code>CreatesSolversFromStringKey</code>
RF2	O método Dormand–Prince 5(4) está disponível e integra EDOs corretamente.	<code>MatchesExponentialDecay...</code> , <code>MatchesHarmonicOscillator...</code>
RF3	É possível escolher qual solver usar em tempo de execução.	<code>...WithDormandPrinceSolver</code>
RF4	O <code>BioNetwork</code> usa o solver selecionado e acerta com qualquer um deles.	<code>...SolversAgreeOnSameModel</code>
RF5	A seleção do solver é persistida no modelo (salvar/carregar).	<code>...RoundTripsThroughPersistence</code>
RF6	Seleção inválida é detectada na validação, sem quebrar a execução.	<code>RejectsUnknownOdeSolverKey</code> ; fallback
RF7	A Factory é extensível a novos solvers sem alterar o cliente (Aberto–Fechado).	<code>RegisterCreatorIsOpenForExtension</code>
RF8	Os solvers respeitam o contrato defensivo (entradas inválidas, passo nulo).	<code>OdeSolverContractTest.*</code>
<i>Parte 2 — Difusão N-D pelo Método das Linhas</i>		
RF9	Resolve a difusão em N dimensões, com N parametrizável.	<code>...RunsInThreeDimensions</code> ; 1-D/2-D/3-D
RF10	A EDP vira sistema de EDOs (MOL) e usa os mesmos solvers da Factory.	<code>BothSolversAgreeOnSameSystem</code>
RF11	O resultado casa a solução analítica (autovetor discreto).	<code>...MatchesAnalyticalEigenmodeDecay</code>
RF12	Contorno de Neumann conserva massa; Dirichlet fixa as bordas.	<code>NeumannConservesMass</code>
RF13	Configuração inválida é rejeitada na validação.	<code>RejectsInvalidConfiguration</code>

3 Fundamentação numérica

3.1 Runge–Kutta de 4^a ordem (RK4) — método legado

Método explícito de 4 estágios e passo fixo, com erro global $O(h^4)$ e custo de exatamente 4 avaliações de f por passo. Não estima o erro local nem adapta o passo: a precisão depende inteiramente do `stepSize` escolhido pelo usuário.

3.2 Dormand–Prince 5(4) — método novo

É um par encaixado (*embedded pair*) de Runge–Kutta explícito de 7 estágios (Dormand & Prince, 1980). Os mesmos estágios $k_1 \dots k_7$ produzem duas estimativas de $y(t_0 + h)$: uma de 5^a ordem

(propagada como solução) e uma de 4ª ordem (usada só para estimar o erro local). A diferença

$$\text{err} = h \sum_i (b_i^{(5)} - b_i^{(4)}) k_i$$

fornece um estimador de erro barato que controla automaticamente o tamanho do passo. O par é **FSAL** (*First Same As Last*): como $c_7 = 1$ e a 7ª linha do tableau coincide com os pesos de 5ª ordem, o último estágio k_7 de um passo aceito é igual ao k_1 do passo seguinte — assim, após o primeiro passo, cada passo aceito custa apenas 6 novas avaliações de f . O controle de passo usa o expoente padrão $1/(4+1) = 0,2$ (a menor ordem do par é 4), um fator de segurança e razões de crescimento/encolhimento limitadas. A norma do erro é a RMS escalada

$$\|\text{err}\| = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(\frac{\text{err}_i}{sc_i} \right)^2}, \quad sc_i = \text{atol} + \text{rtol} \cdot \max(|y_i|, |y_i^*|).$$

3.3 Conciliação com o contrato `OdeSolver_if`

O contrato expõe um único macro-passo fixo `advance(system, t0, dt, y0, y1)`. Para preservar esse contrato e aproveitar o controle de erro, o Dormand–Prince integra de t_0 a $t_0 + dt$ usando um ou mais **sub-passos adaptativos internos**, devolvendo o estado exatamente em $t_0 + dt$. O cliente continua usando a mesma malha temporal de antes, mas ganha precisão adaptativa dentro de cada macro-passo.

4 Projeto de software

4.1 Decisões de engenharia

- **Factory baseada em registro (não switch rígido).** Solvers internos são registrados numa tabela `map<string, Creator>`. Novos métodos podem ser adicionados estendendo o `enum/tabela` ou, sem tocar na Factory, chamando `registerCreator` na inicialização — concretizando o **Princípio Aberto–Fechado**.
- **Dois espaços de chave.** Um `enum OdeSolverType` (type-safe, para código) e uma chave *string* estável (para persistência em arquivo de modelo e controles de GUI), mantidos em sincronia por `toKey/fromKey`.
- **Propriedade clara da memória.** `create()` devolve `std::unique_ptr<OdeSolver_if>` — sem vazamentos, sem dono ambíguo.
- **Header-only.** Acompanha o estilo já usado pelos solvers do repositório, então o componente integra *sem mudança no sistema de build*. O registro é um singleton de Meyers (estático local de função), bem definido entre unidades de tradução.
- **Solver sem estado entre chamadas.** `advance` é `const` e toda a memória de trabalho é local; uma única instância pode ser compartilhada com segurança.
- **Compatibilidade retroativa.** Os internos do RK4 não mudaram; o default é "RungeKutta4"; chave vazia/legada resolve para o default; chave desconhecida cai em RK4 (com aviso na validação), de modo que modelos antigos continuam executando.

4.2 Diagrama de classes (UML) — Parte 1

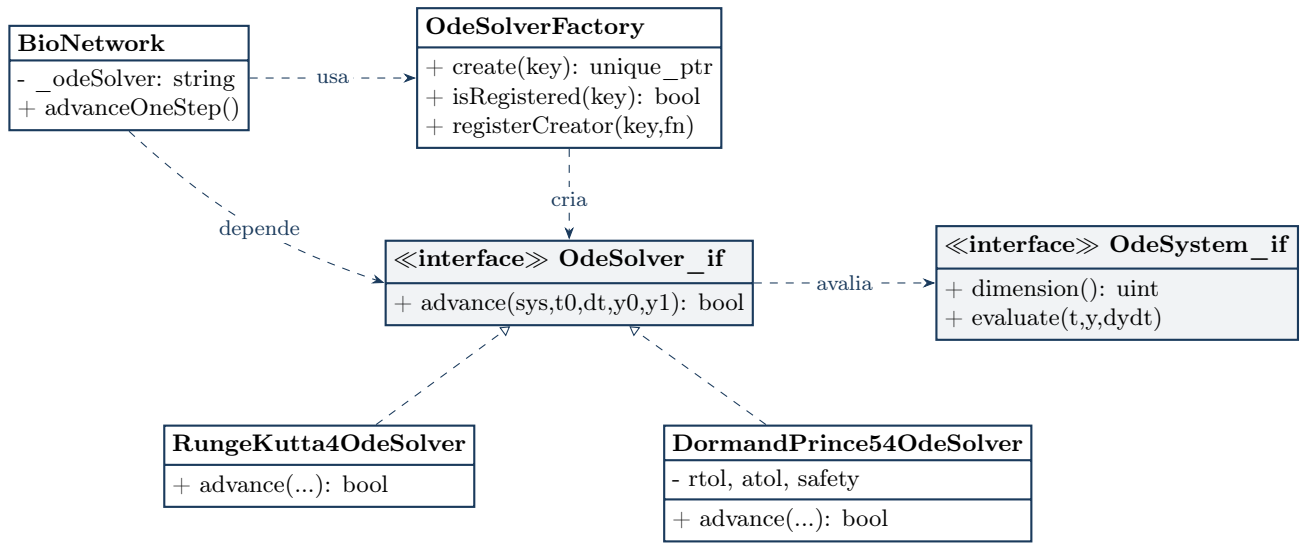


Figura 1: As implementações realizam `OdeSolver_if`; a Factory as cria; `BioNetwork` depende apenas das abstrações `OdeSolver_if` e `OdeSystem_if`.

4.3 Diagrama de sequência (UML) — `advanceOneStep`

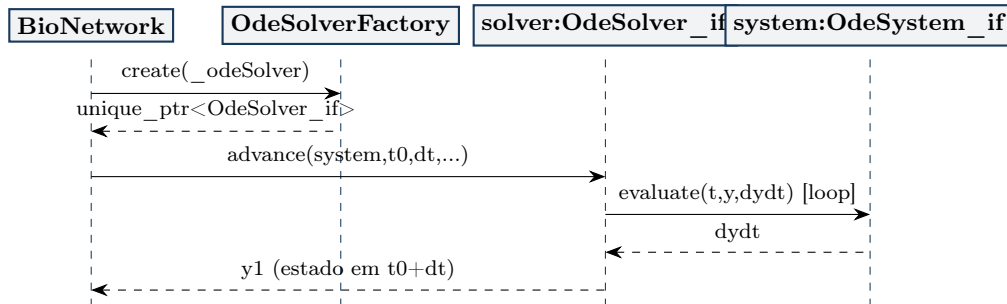


Figura 2: O solver é escolhido em tempo de execução pela chave `_odeSolver` e o macro-passo é delegado ao solver concreto, que avalia o sistema uma ou mais vezes.

5 Implementação

5.1 Arquivos do componente

Arquivo	Papel	Situação
<code>source/tools/OdeSolverFactory.h</code>	Factory (registro, enum/chave, extensão)	novo
<code>source/tools/DormandPrince54OdeSolver.h</code>	Solver DP5(4) adaptativo	novo
<code>source/tools/RungeKutta4OdeSolver.h</code>	Solver RK4 (legado)	doc; algoritmo intacto
<code>.../BiochemicalSimulation/BioNetwork.{h, cpp}</code>	Seleção de solver, persistência, validação	modificado

5.2 Trecho-chave: a Factory

```

std::unique_ptr<OdeSolver_if> OdeSolverFactory::create(const std::string& key) {
    auto& reg = registry();

```

```

auto it = reg.find(key);
if (it != reg.end()) return it->second(); // cria o solver registrado
return std::make_unique<RungeKutta4OdeSolver>(); // fallback defensivo (RK4)
}

```

5.3 Trecho-chave: integração no BioNetwork

O ponto de acoplamento foi substituído. Em vez de instanciar RK4 diretamente, o runner depende da abstração e da chave configurável:

```

// antes: RungeKutta4OdeSolver solver;
std::unique_ptr<OdeSolver_if> solver = OdeSolverFactory::create(_odeSolver);
if (!solver || !solver->advance(system, _currentTime, dt, y0.data(), y1.data())) {
    _lastErrorMessage = "... failed to advance the ODE system with solver \"" + _odeSolver + "\".";
    ;
    return false;
}

```

O campo `_odeSolver` (default "RungeKutta4") é exposto como `SimulationControl` (editável na GUI), persistido em `_loadInstance/_saveInstance` e validado em `_check` contra `OdeSolverFactory::isRegistered`.

5.4 Trecho-chave: o núcleo adaptativo do DP5(4)

```

if (errNorm <= 1.0) { // aceita o sub-passo
    t += h; y = yNext;
    firstSameAsLast = true; // k7 vira o proximo k1 (FSAL)
    h *= scaleFactor(errNorm); // cresce o passo
} else { // rejeita e tenta menor
    h *= scaleFactor(errNorm);
    if (h < stepFloor(t)) return false; // underflow: nao atinge a tolerancia
}

```

6 Estratégia de testes

Dois níveis, ambos automatizados em GoogleTest no build do repositório, e também reproduzidos em um *harness* autônomo (sem Qt/CMake) para verificação numérica rápida do núcleo.

- **Unitários (núcleo, ligam só genesys_tools):** `test_tools_ode_solver_factory.cpp` (14 casos) — contrato da Factory, corretude do DP5(4) contra soluções fechadas, ordem de convergência do RK4 e casos de borda do contrato; `test_tools_diffusion_mol.cpp` (9 casos) — núcleo MOL.
- **Integração (com o kernel, usam os *probes* existentes):** blocos adicionados a `test_simulator_runtime.cpp` — executam `BioNetwork` (5 casos) e `DiffusionField` (7 casos) com cada solver, comparam os métodos no mesmo modelo, verificam persistência e rejeição de chave/configuração inválida.
- **Harness autônomo:** `standalone_validation.cpp` (28 *checks*), `diffusion_validation.cpp` (20 *checks*) e `benchmark.cpp` — compilam só com `g++ -std=c++23`, sem Qt, para verificação numérica direta.

7 Resultados — Parte 1

Valores reproduzidos para este relatório (Apple clang 17, C++23). A íntegra das saídas está em `docs/resultados_testes.txt`.

7.1 Corretude contra soluções analíticas

Problema	Método	malha	erro
Decaimento exponencial $y' = -0,7y$	RK4	30	$2,7 \times 10^{-7}$
Decaimento exponencial	DP5(4)	30	$6,8 \times 10^{-10}$
Oscilador harmônico $\omega = 2$	RK4	200	$1,6 \times 10^{-8}$
Oscilador harmônico	DP5(4)	50	$1,6 \times 10^{-9}$
Crescimento logístico	RK4	100	$2,5 \times 10^{-6}$
Crescimento logístico	DP5(4)	25	$2,8 \times 10^{-6}$

7.2 Ordem de convergência empírica do RK4

Dobrando o número de passos, o erro cai por um fator ≈ 16 , confirmando a 4ª ordem:

passos	erro	ordem observada
10	$4,27 \times 10^{-6}$	—
20	$2,45 \times 10^{-7}$	4,12
40	$1,47 \times 10^{-8}$	4,06
80	$9,00 \times 10^{-10}$	4,03

7.3 Acurácia vs. custo (crescimento logístico, $y(8)$)

RK4 com passo fixo vs. DP5(4) num único macro-passo subdividido adaptativamente. O custo é medido em avaliações de f (fator dominante); $y(8)$ exato = 99,9392092753.

Configuração	erro global	avaliações de f
RK4 — 32 passos	$9,1 \times 10^{-5}$	128
RK4 — 128 passos	$3,2 \times 10^{-7}$	512
DP5(4) — rtol 1×10^{-5}	$7,8 \times 10^{-5}$	139
DP5(4) — rtol 1×10^{-7}	$1,3 \times 10^{-6}$	313
DP5(4) — rtol 1×10^{-9}	$1,7 \times 10^{-8}$	715

Leitura: o DP5(4) permite especificar a *precisão desejada* (tolerância) em vez de adivinhar o passo, ajustando o esforço automaticamente. Em problemas suaves a vantagem é a robustez (precisão garantida sem *tuning* manual); em transientes rápidos, a adaptação concentra esforço onde a dinâmica é mais intensa.

7.4 Validação pelo BioNetwork (reação reversível $A \rightleftharpoons B$)

Com $k_f = 0,1$, $k_r = 0,05$, $A_0 = 10$, o equilíbrio analítico em $t = 1$ é $A = 9,0714$, $B = 0,9286$. Ambos os solvers atingem o resultado; o DP5(4) usa malha **20× mais grossa** (0,05 vs. 0,001):

Chave de solver	passo	erro em $A(1)$	conservação de massa
RungeKutta4	0,001	$5,3 \times 10^{-15}$	ok (7×10^{-15})
DormandPrince54	0,05	$8,9 \times 10^{-15}$	ok (4×10^{-15})
chave desconhecida → fallback RK4	0,001	$5,3 \times 10^{-15}$	ok

Síntese (Parte 1). **42 verificações automatizadas, 0 falhas** (28 no harness numérico do núcleo + 14 nos testes GoogleTest do componente), além dos 5 blocos de integração com o **BioNetwork**. A ordem de convergência do RK4 foi confirmada ($\approx 4,0$) e o DP5(4) entrega a mesma precisão com malha substancialmente mais grossa.

8 Extensão: plugin de difusão N -D pelo Método das Linhas

A extensão exercita e valida a arquitetura da Parte 1 num domínio diferente: em vez de cinética química, resolve uma **equação diferencial parcial (EDP)** de difusão, reaproveitando integralmente a `OdeSolverFactory` e os dois solvers.

8.1 Problema e Método das Linhas

Resolve a equação de difusão (calor) em N dimensões, com N parametrizável:

$$\frac{\partial u}{\partial t} = D \nabla^2 u.$$

O **Método das Linhas (MOL)** discretiza apenas o espaço, mantendo o tempo contínuo: o Laplaciano em cada nó da malha é aproximado por diferença finita central de 2^a ordem, somada sobre todas as dimensões,

$$\nabla^2 u \approx \sum_{d=1}^N \frac{u_{\dots, i_d+1, \dots} - 2u_{\dots, i_d, \dots} + u_{\dots, i_d-1, \dots}}{h_d^2}.$$

Isso converte a EDP num **sistema de EDOs** $\dot{y} = D L y$ (uma equação por nó), então integrado no tempo por **qualquer** `OdeSolver_if` obtido da Factory — exatamente o mesmo *pipeline* da Parte 1. A dimensionalidade N é o tamanho do vetor de forma da malha; o armazenamento é *row-major* com *strides* pré-computados, de modo que o mesmo código serve 1-D, 2-D, 3-D e além.

8.2 Condições de contorno

Dirichlet (valor fixo): nós de fronteira são mantidos constantes ($\dot{y} = 0$). **Neumann** (fluxo nulo): um nó-fantasma refletido anula a derivada normal, tornando a segunda diferença na face igual a $2(u_{\text{interno}} - u_{\text{fronteira}})/h_d^2$. Sob Neumann, a massa total (integral trapezoidal, com nós de fronteira de peso $\frac{1}{2}$ por face) é conservada à precisão de máquina.

8.3 Projeto (UML) — Parte 2

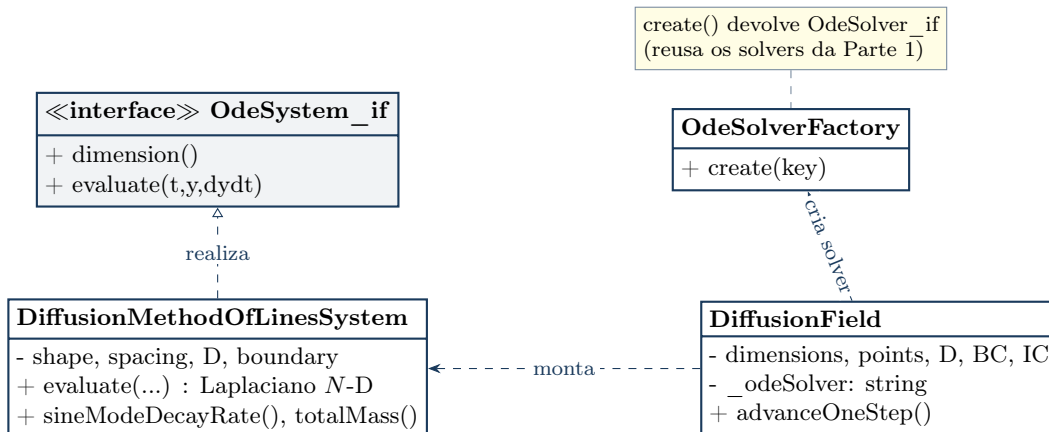


Figura 3: `DiffusionMethodOfLinesSystem` realiza `OdeSystem_if`; o plugin `DiffusionField` constrói esse sistema e o integra com um solver da *mesma* `OdeSolverFactory` da Parte 1.

O plugin `DiffusionField` espelha a estrutura do `BioNetwork` (`SimulationControls`, persistência, `_check`, `simulate/advanceOneStep`). Parâmetros configuráveis: `Dimensions` (N), `PointsPerDimension`, `DomainLength`, `DiffusionCoefficient`, `BoundaryCondition`, `InitialCondition` (modo senoidal/gaussiana/constante) e `OdeSolver` (a mesma chave da `Factory`). A geometria é constante no tempo, então cada passo reconstrói apenas o descritor leve do sistema e reutiliza o estado de campo.

8.4 Estratégia de validação

A chave da validação rigorosa é uma propriedade espectral: o produto de senoides de grade $\prod_d \sin(k_d \pi x_d / L_d)$ é um **autovetor exato do Laplaciano discreto N -D**, decaindo como $e^{-\lambda t}$ com taxa discreta exata

$$\lambda = D \sum_d \frac{4}{h_d^2} \sin^2\left(\frac{k_d \pi}{2(n_d - 1)}\right).$$

Com um solver apertado, o campo simulado deve casar esse decaimento à tolerância do solver, em qualquer dimensão.

8.5 Resultados (reproduzidos)

Verificação	resultado	significado
Decaimento do autovetor 1-D (DP5(4))	erro $1,1 \times 10^{-9}$	Laplaciano 1-D correto
Decaimento do produto 2-D	erro $6,6 \times 10^{-9}$	Laplaciano 2-D correto
Decaimento do produto 3-D	erro $8,1 \times 10^{-8}$	N parametrizável ($N > 2$)
Ordem de convergência espacial	$\approx 2,00$	diferença central $O(h^2)$ confirmada
Conservação de massa (Neumann)	$8,9 \times 10^{-16}$	fluxo nulo correto (precisão de máquina)
RK4 <i>vs.</i> DP5(4) no mesmo MOL	$7,3 \times 10^{-14}$	solvers concordam

Estabilidade (conexão com a Parte 1). A difusão é *stiff* em malhas finas: o passo explícito estável escala com h^2 . Numa malha de 101 nós com condição inicial gaussiana (rica em frequências) e um **único macro-passo**, o RK4 fixo ultrapassa o limite de estabilidade e **explode** (erro-máx $3,4 \times 10^2$, L_2 $8,8 \times 10^2$ *vs.* referência 1,8), enquanto o **DP5(4) adapta** os sub-passos e permanece exato (erro-máx $3,1 \times 10^{-8}$, L_2 casando a referência). Esse contraste demonstra concretamente, num problema de EDP, por que a escolha de solver habilitada pela `Factory` importa.

Validação do plugin. O núcleo MOL roda em harness autônomo (20 verificações) e em `GoogleTest` (9 casos), todos passando. O plugin `DiffusionField.cpp` foi **compilado para objeto contra o kernel real** (zero erros/warnings), e a superfície de API dos testes de integração foi verificada por *syntax-check* contra o kernel — só falta linkar o executável Qt completo, que exige o ambiente local.

9 Reprodução e ambiente

Os núcleos numéricos das duas partes **não dependem do GenESyS** e foram recompilados e reexecutados para este relatório. As interfaces `OdeSolver_if.h/OdeSystem_if.h` pertencem ao simulador; para a verificação *off-tree* basta tê-las no *include path*.

```
# Parte 1 (28 checks) e Parte 2 (20 checks) + benchmark
INC="-I source -I source/tools"
g++ -std=c++23 $INC tests/standalone_validation.cpp -o validate && ./validate
g++ -std=c++23 $INC tests/diffusion_validation.cpp -o diff && ./diff
g++ -std=c++23 $INC tests/benchmark.cpp -o bench && ./bench
```

Resultado da reprodução (Apple clang 17, C++23): `standalone_validation` $\Rightarrow$ **checks: 28, failures: 0**; `diffusion_validation` $\Rightarrow$ **checks: 20, failures: 0**; `benchmark` reproduz a

tabela acurácia/custo idêntica à documentada. Os 35 casos GoogleTest (14 Factory + 9 MOL + 5 BioNetwork + 7 DiffusionField) e a validação do caminho do BioNetwork executam no *build* completo do simulador (Qt/CMake), por dependerem de `MassActionOdeSystem.h` e do kernel.

10 Instruções de implantação (deployment)

Compatível com a IDE QtCreator e com o *build* CMake do repositório.

Parte 1 — copiar (novos) e aplicar os patches:

```
# copiar para a arvore do simulador:
source/tools/OdeSolverFactory.h
source/tools/DormandPrince54OdeSolver.h
source/tests/unit/test_tools_ode_solver_factory.cpp
# modificar os arquivos existentes via patches:
cd Genesys-Simulator
git apply patches/RungeKutta4OdeSolver.h.patch
git apply patches/BioNetwork.h.patch
git apply patches/BioNetwork.cpp.patch
```

Parte 2 — copiar (novos):

```
source/tools/DiffusionMethodOfLinesSystem.h
source/plugins/data/DiffusionSimulation/ # diretorio do plugin
source/tests/unit/test_tools_diffusion_mol.cpp
```

O `.cpp` do plugin é captado automaticamente pelo `GLOB_RECURSE` de `plugins/data/CMakeLists.txt` (sem edição).

Testes: aplique `patches/CMakeLists.unit.add.txt` em `CMakeLists.txt` dos testes e cole os blocos de `bionetwork_solver_tests.snippet.cpp` e `diffusionfield_tests.snippet.cpp` em `test_simulator_runtime.cpp` (após os *probes*). Compile (QtCreator/CMake) e rode `ctest`.

Uso no modelo. Em um BioNetwork, defina o controle OdeSolver com "RungeKutta4" (default) ou "DormandPrince54". Em um DiffusionField, defina Dimensions (N), PointsPerDimension, DomainLength, DiffusionCoefficient, BoundaryCondition (Dirichlet/Neumann), InitialCondition (SineMode/Gaussian/Constant) e OdeSolver. A escolha é salva no arquivo do modelo e validada no `_check`.

11 Análise crítica e limitações

- O DP5(4) é um método **explícito**: para sistemas muito *stiff*, métodos implícitos seriam mais eficientes. A arquitetura já está pronta — basta registrar um novo solver na Factory, sem alterar o BioNetwork nem o DiffusionField.
- O passo inicial do DP5(4) é o macro-passo inteiro; em transientes muito rápidos isso gera algumas rejeições iniciais até o controlador ajustar. Um estimador de passo inicial (heurística de Hairer) reduziria esse custo — melhoria futura de baixo risco.
- A tolerância do DP5(4) usa *defaults* razoáveis. Expô-la como controle adicional dos plugins é uma extensão natural (o construtor já é parametrizável).
- O *fallback* silencioso para RK4 em chave desconhecida favorece robustez; a sinalização ao usuário ocorre na validação (`_check`), combinando segurança e *feedback*.
- **Difusão (Parte 2):** o esquema espacial é de 2ª ordem e a difusão explícita é *stiff* em malhas finas; o DP5(4) adapta-se, mas para malhas muito grandes um método **implícito** seria mais econômico — e, de novo, basta registrá-lo na Factory. Limitações atuais do plugin: malha uniforme e isotrópica (mesmo número de pontos e comprimento por eixo) e ausência de termo-fonte; ambas são extensões diretas (a classe do sistema já aceita espaçamento por dimensão).

12 Conclusão

O DCS cumpre os requisitos em duas partes coesas. Na Parte 1, a Factory desacopla o simulador do método numérico, o Dormand–Prince 5(4) adiciona controle de erro adaptativo, a escolha do solver é feita em tempo de execução e persistida, e a validação pelo BioNetwork confirma corretude e eficiência. Na Parte 2, a mesma arquitetura é reaproveitada para resolver a equação de difusão em N dimensões pelo Método das Linhas: a EDP vira um sistema de EDOs integrado pelos solvers existentes, com validação contra autovetores discretos exatos, ordem espacial $O(h^2)$ e conservação de massa. Em ambas, as decisões de engenharia (registro extensível, propriedade explícita, compatibilidade retroativa, código *header-only* para os núcleos) priorizam clareza, segurança e baixo impacto de integração. A extensão é a melhor evidência do valor da arquitetura: **um problema inteiramente novo (uma EDP) foi resolvido sem alterar uma linha do componente de solvers.**

Referências

- Dormand, J. R.; Prince, P. J. (1980). *A family of embedded Runge–Kutta formulae*. Journal of Computational and Applied Mathematics, 6(1), 19–26.
- Hairer, E.; Nørsett, S. P.; Wanner, G. (1993). *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer.
- Gamma, E.; Helm, R.; Johnson, R.; Vlissides, J. (1994). *Design Patterns* (Factory Method). Addison–Wesley.
- Repositório base GenESyS, branch 2026-1.